

University of Asia Pacific
Department of Computer Science and Engineering

CSE 402: Operating System Lab
Lab Report: 02

Report Title: SJF and FCFS Implementation, Comparison with FCFS and SJF

Submitted by:

Name: Faiaz Masnun Labib
Student ID: 23101128
Section: C-1

Submitted to:

Atia Rahman Orthi
Lecturer, Dept. of CSE, UAP
Date of Submission: 20/08/2026

1 Problem Statement

Develop a Python program to simulate and compare the **First Come First Serve (FCFS)** and **Shortest Job First (SJF)** non-preemptive CPU scheduling algorithms using the following set of processes:

Process ID	Arrival Time (AT)	Burst Time (BT)
P1	3	3
P2	2	5
P3	5	4
P4	1	3
P5	6	2

For each scheduling algorithm, calculate the **Completion Time (CT)**, **Turnaround Time (TAT)**, and **Waiting Time (WT)** for every process. Also determine the execution sequence, Average Turnaround Time, and Average Waiting Time for both FCFS and SJF. Finally, compare the results of the two algorithms and determine which scheduling algorithm performs better for the given dataset in terms of average waiting time and average turnaround time.

2 Source Code

```
1 pro = [  
2     ["P1", 3, 5],  
3     ["P2", 2, 4],  
4     ["P3", 4, 3],  
5     ["P4", 1, 2],  
6     ["P5", 5, 3]  
7 ]  
8  
9 remaining_pro = pro.copy()  
10  
11 time = 0  
12 total_tat = 0  
13 total_wt = 0  
14 execution_order = []  
15  
16 print("PID\tAT\tBT\tCT\tTAT\tWT")  
17  
18 while remaining_pro:  
19     available_pro = [p for p in remaining_pro if p[1] <= time]  
20  
21     if not available_pro:  
22         time = min(remaining_pro, key=lambda x: x[1])[1]  
23         continue  
24  
25     chosen = min(available_pro, key=lambda x: (x[2], x[1]))  
26     remaining_pro.remove(chosen)  
27  
28     pid, at, bt = chosen  
29     ct = time + bt  
30     tat = ct - at  
31     wt = tat - bt  
32  
33     total_tat += tat  
34     total_wt += wt  
35     execution_order.append(pid)  
36  
37     print(f"{pid}\t{at}\t{bt}\t{ct}\t{tat}\t{wt}")  
38     time = ct  
39  
40 n = len(pro)  
41 print(f"\nAverage TAT = {total_tat / n:.2f}")  
42 print(f"Average WT = {total_wt / n:.2f}")  
43  
44 print("\nExecution Order (Gantt Chart):")  
45 for pid in execution_order:  
46     print(pid, end=" --> ")  
47 print("End")
```

GitHub Repository Link: [Click Here](#)

3 Simulation Output

```

===== SJF =====

Pd  AT  BT  CT  TAT  WT
P4  1   3   4   3   0
P1  3   3   7   4   1
P5  6   2   9   3   1
P3  5   4  13   8   4
P2  2   5  18  16  11

Execution Sequence:
P4 P1 P5 P3 P2

Average TAT = 6.8
Average WT = 3.4

===== FCFS =====

Pd  AT  BT  CT  TAT  WT
P4  1   3   4   3   0
P2  2   5   9   7   2
P1  3   3  12   9   6
P3  5   4  16  11   7
P5  6   2  18  12  10

Execution Sequence:
P4 P2 P1 P3 P5

Average TAT = 8.4
Average WT = 5.0

===== COMPARISON =====

Algorithm  Avg TAT  Avg WT
-----
SJF        6.80    3.40
FCFS       8.40    5.00

SJF has lower Average Turnaround Time.
SJF has lower Average Waiting Time.

Result: SJF is Better than FCFS.

```

Figure 1: Python Simulation Output for FCFS vs SJF CPU Scheduling

4 Results and Performance Analysis

Why is SJF better than FCFS in this case?

In this problem, **SJF** is better than **FCFS** because it selects the shortest available process first, which helps reduce the overall waiting time of the processes. In FCFS, processes are executed only according to their arrival time, so a process with a longer burst time may cause shorter processes to wait for a long time.

For example, P_2 has a burst time of 5 units and is executed before some shorter processes in FCFS simply because it arrived earlier. In SJF, shorter processes such as P_1 and P_5 are executed before longer processes whenever they are available. As a result, more processes finish earlier and the average waiting time and turnaround time become lower. In this dataset:

- **Average Waiting Time:** SJF = 3.4 vs FCFS = 5.0
- **Average Turnaround Time:** SJF = 6.8 vs FCFS = 8.4

Therefore, SJF gives better performance than FCFS for this particular set of processes.

5 Discussion & Analysis

1. Why SJF Achieves Shorter Average Execution Times Although both algorithms process the identical set of arrival and burst times (resulting in the same total workload of 18 time units), **non-preemptive SJF** minimizes queue congestion:

- **Mitigating the Convoy Effect:** FCFS executes long jobs first if they arrive early (e.g., P_2 with $BT = 5$), forcing multiple shorter jobs (P_1, P_5) to wait in the ready queue and inflating their individual WT and TAT.
- **Optimal Throughput:** SJF prioritizes available jobs with the smallest burst times, flushing short tasks rapidly and lowering the cumulative waiting sum across the system.

2. Trade-offs and Real-World Applications

- **FCFS Applications:** Ideal for batch processing, print queues, FIFO buffers, and real-time transaction processing where strict order fairness, predictability, and zero scheduling overhead are essential.
- **Trade-off:** While SJF maximizes overall CPU efficiency, it introduces potential *starvation* for compute-heavy processes (like P_2), whereas FCFS guarantees starvation-free execution at the cost of higher average latency.